DENSITY-BASED CLUSTERING AND CONTEXTUAL GRAPH AUTOENCODERS FOR ANOMALY DETECTION IN INDUSTRIAL AND SATELLITE TELEMETRY

Alessandro Bevilacqua
Department of Computer Science
University of Bari Aldo Moro
Bari, Italy
a.bevilacqua11@studenti.uniba.it

November 6, 2026

ABSTRACT

Multivariate telemetry from industrial plants and spacecraft comprises large, complex datasets, where anomaly detection represents one of the most critical and highly requested tasks. This work reimplements DENCAST, a distributed density-based clustering algorithm built on locality-sensitive hashing, in PySpark, and extends it into an anomaly detector, DENCAD, in which a graph autoencoder reconstructs the centre of the contextual subgraph a new instance is attached to. DENCAD is compared with four other detectors on the same data and under the same protocol: the *plain autoencoder*, the same network on one observation in isolation (graph-free); the *graph autoencoder*, the same network over the L-hop neighbourhood the graph defines, without any clustering; *DENCAST distance*, the distance to the nearest clustered object; and *K-Means*, the distance to the nearest centroid. All five produce a score and share one thresholding rule, a multiple of the training deviation whose factor is chosen on validation (sigma rule); the comparison itself rests on average precision and ROC-AUC, which are threshold-free, so they reflect the ranking each detector produces rather than where its threshold happens to fall. From the results, it is quite clear that the autoencoder detectors are generally ahead of the distance-based ones, K-Means and DENCAST distance, even if not so much. Among the autoencoders no clear winner emerges, and the neighbourhood graph does not always justify its overhead. However, it is worth noting that DENCAD comes first on both ESA variants (normal and contaminated) that count rare events as legitimate when considering AP as the metric.

1 Introduction

Industrial control systems and spacecraft emit continuous multivariate telemetry whose faults are rare but costly. Detection is posed as a one-class problem: a model of normal behaviour is fitted on a period believed to be clean and later instances are scored by how far they fall from it. Three properties make the setting challenging: the data is large — the ESA mission used here holds seven million readings over fourteen years — so the neighbourhood graph can be very expensive to compute; the detector must be *inductive*, scoring an instance that arrives after training without refitting anything; and an anomaly is not always a property of one reading but often a trend across many.

DENCAST [1] addresses the first two challenges for clustering. It approximates a neighbourhood graph with locality-sensitive hashing, identifies core objects by their degree in the DBSCAN sense [2] and propagates cluster labels over the graph, all in Spark. Furthermore, a new object is assigned to an existing cluster by similarity to a clustered one rather than by refitting.

The third challenge is addressed in the preprocessing stage of this work. Readings are grouped into fixed, non-overlapping windows of a set length, and each window is reduced to summary statistics per feature: mean, standard

Table 1: Datasets after preprocessing and the windows they produce. *Period* is the sampling interval and *steps* the readings a complete window holds; d is the number of window features. Positives and segments are counted on the test split, a segment being a run of consecutive positive windows.

	source				windows				
	rows	sensors	period	steps	d	train	valid.	test	pos./seg.
SWaT	946,719	52	1 s	60	124	8,280	3,749	3,750	728 / 13
HAI	802,800	87	1 s	30	220	19,080	3,840	3,840	169 / 19
ESA	7,364,160	76	60 s	132	216	34,576	8,509	8,629	937 / 24

deviation, minimum and maximum for the numeric channels, and mode, entropy, distinct count and largest share for the categorical ones. The window is then the object that is hashed, clustered and scored, and it is labeled as positive if any of its constituent readings is positive (i.e. anomalous).

DENCAD constructs the contextual subgraph for a new window by retrieving its nearest clustered windows, and employs a graph autoencoder to reconstruct the target window from this context, using the reconstruction error as the anomaly score.

Our main contributions are a PySpark reimplementation of DENCAST starting from the original implementation written in Scala; DENCAD, an inductive detector over contextual subgraphs using density-based clustering; and an evaluation protocol applying the same thresholding rule and calibration procedure to five detectors.

2 Related Work

Distributed density-based clustering. DBSCAN [2] defines clusters by density, but its neighbourhood queries parallelise poorly. DENCAST [1] replaces exact neighbourhoods with LSH candidates and label propagation in order to adopt a design that fits Spark's shuffle model. We keep its graph construction and clustering. Its hash family, random hyperplanes [3], is canonical for angular similarity and invariant to the norm of the vector; since our features are individually standardised, that norm represents the aggregate deviation from normal operation—precisely the signal we want to preserve. We therefore adopt the p-stable family of Datar et al. [4] (the family of E2LSH), whose collision probability decreases with Euclidean distance.

Graph autoencoders for anomaly detection. DOMINANT [6] reconstructs both attributes and structure of an attributed graph with a graph convolutional encoder [7] and ranks nodes by reconstruction error. It is transductive: a new node requires recomputing the graph. Inductive aggregation [8] removes that constraint in principle; we obtain it by construction, attaching the new window to a frozen graph and reading only its own receptive field. Containing the cost of a full-graph convolution by partitioning is the argument of Cluster-GCN [9]; our contextual subgraphs exploit the same economy, with partitions derived come from the density clustering rather than from a graph partitioner.

3 Materials

Three public datasets are used, summarised in Table 1, where the ESA positive label can combine anomalies and rare events (i.e., rare events can be considered either legitimate or anomalous). SWaT is a water treatment testbed with labelled attack campaigns [10]; HAI 23.05 is a hardware-in-the-loop industrial security dataset [11]; ESA is Mission 1 of the ESA Anomaly Dataset [12], comprising fourteen years of satellite telemetry whose 200 annotations distinguish 118 anomalies, 78 *rare events* (namely, unusual but legitimate behaviour), and 4 communication gaps. Splits are chronological, so validation and test follow the training period with no overlapping windows.

Cleaning. Rows with no reading in any feature column are dropped, as are rows identical to an earlier one including the timestamp; both counts are zero on all three datasets. Columns missing at least 30% of the training rows are also discarded, which removes seven channels on SWaT that are entirely absent during training, together with columns constant on the training split and one of every pair whose absolute Pearson correlation exceeds 0.99, or Cramér's V for categorical pairs. Remaining gaps are imputed with the training median, or the mode for categorical columns.

Windowing. Readings are aggregated into non-overlapping windows of a fixed number of readings, which serve as the unit that is scored and labelled: a window is positive if any of its readings are. Numeric features contribute their mean, standard deviation, minimum, and maximum, while categorical features contribute their mode, entropy, distinct

count, and largest share. Windows holding fewer than half the expected readings are dropped, which on ESA removes 85 of the 34,661 training windows and one window in each of the validation and test splits. Additionally, window features are individually standardised.

One-class training and the two ESA labels. The training split is rendered one-class by removing its positive instances, ensuring that the metrics upon which detectors calibrate are not inflated by anomalies. (Since the SWaT and HAI training splits are already anomaly-free, no removal was required.) For ESA, a second variant retains these positive instances to establish a contaminated setting, where the detector is trained on data containing anomalies, allowing us to evaluate the masking effect caused by inflated baseline deviations. Furthermore, ESA annotations admit two interpretations of the positive class—wherein rare events are either treated as anomalies or grouped with nominal behaviour. Crossing these two labelling strategies with the two training configurations yields four distinct variants for ESA. Under the combined label, the validation split contains 882 positive windows across 27 segments and the test split contains 937 across 24; under the anomaly-only label, the validation split contains 294 positive windows across 17 segments and the test split contains 136 across 15.

4 Methods

4.1 Neighbourhood graph

Each window is a vector $x \in \mathbb{R}^d$ holding its d features, and it receives a signature of b integers, one for each of b random directions. The i-th direction is a vector $a_i \in \mathbb{R}^d$ of independent standard normal entries; projecting x on it gives the scalar $a_i \cdot x$; that line is cut into buckets of width w, shifted by a random offset c_i; and the integer $h_i(x)$ says which bucket x falls into,

$$h_i(x) = \left\lfloor \frac{a_i \cdot x + c_i}{w} \right\rfloor, \qquad a_i \sim \mathcal{N}(0, I), \quad c_i \sim \mathcal{U}[0, w), \tag{1}$$

so the signature of x is $(h_1(x), \ldots, h_b(x))$. This is the p-stable construction with $p = 2$: the projections of two windows differ by a Gaussian whose deviation is their euclidean distance, so the chance of sharing a bucket falls with that distance. The bucket width is $w = \texttt{width_factor} \cdot \varepsilon$, tied to the radius of interest.

Candidates are generated without exact bucket equality, by the permuted sorted-neighbourhood scheme of Ravichandran et al. [5] that DENCAST itself uses: the order of the b values is permuted, all windows are sorted lexicographically by the permuted signature, and each is paired with the `band` windows that follow it over several permutations. Every candidate pair is then verified exactly and an edge is kept when $\|x_u - x_v\| \leq \varepsilon$. The hash therefore decides only which pairs are compared, never which are connected, and the only possible error is a near pair that no permutation proposed.

4.2 Clustering

A node is a *core* object when its degree reaches `min_pts`. Clusters are the connected groups of cores, grown by max-identifier label propagation: a non-core node that reaches a core joins the cluster as a *border* node, and a node that reaches no core remains noise. The propagation stops when the changes fall below `label_change_rate` times the number of directed edges, each undirected edge counting twice.

Calibrating the radius. Unlike a cosine threshold, ε has no universal range, so it is set from the data, and testing quantiles of the pairwise distance distribution is one way to do it.

4.3 Contextual subgraphs and DENCAD

To find the contextual graph of a window to be scored (query), it is necessary to locate its closest node among the clustered ones (i.e., excluding noise points) and retrieve the corresponding cluster. This search can be done by an exact blocked argmin using Spark: queries and pool are split into blocks, each pair of blocks is reduced in a `pandas` task, and the per-query minima are combined.

Step 1: the problem. A query is a vector $x_q \in \mathbb{R}^d$ and the pool is a set P of n_p clustered objects, each with its own vector. The placement looks for

$$\pi(q) \;=\; \arg\min_{p \in P} \|x_q - x_p\|. \tag{2}$$

Step 2: the naive computation. Build the full matrix $D \in \mathbb{R}^{n_q \times n_p}$ with $D_{qp} = \|x_q - x_p\|$ and take the minimum of each row.

Step 3: split the pool. Partition P into s disjoint blocks, $P = P_1 \sqcup P_2 \sqcup \cdots \sqcup P_s$, and compute for each block the partial minimum of a query,

$$m_j(q) \;=\; \min_{p \in P_j} \|x_q - x_p\|. \tag{3}$$

Step 4: Minimum combination The minimum over a disjoint union is the minimum of the partial minima,

$$\min_{p \in P} \|x_q - x_p\| \;=\; \min_{j=1,\ldots,s} m_j(q), \tag{4}$$

Step 5: split the queries as well. With $Q = Q_1 \sqcup \cdots \sqcup Q_r$ there are $r \times s$ block pairs (Q_i, P_j), and each is independent of the others.

Step 6: map and reduce. Each of the $r \cdot s$ pairs is computed on its own and emits $m_j(q)$ for every $q \in Q_i$; the results are grouped by q and reduced by taking the minimum of its s values.

Its contextual subgraph is induced from the cluster of the identified nearest node. Specifically, the query is attached to both the nearest object and any cluster members within distance ε. The subgraph is then formed by extending this attachment set along the cluster's existing edges up to an $(L-1)$-hop neighbourhood, which gives L graph convolutions their complete receptive field. The final graph comprises these nodes, the cluster's edges among them, the query's attachment edges, and the query itself designated as the central target node.

The encoder is a graph convolution with the symmetric normalisation of [7],

$$H^{(l+1)} = \sigma\Big(\hat{A} H^{(l)} W^{(l)}\Big), \qquad \hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}, \tag{5}$$

and the decoder reconstructs the centre's own attributes. The score of a window is the distance between the centre and its reconstruction.

$$s(x) = \sqrt{\tfrac{1}{d} \|\hat{x} - x\|^2}, \tag{6}$$

4.4 Training examples

Training examples are constructed as follows. Each training window—including noise points—is treated as a query and excluded from its own context. It is attached to the contextual subgraph using the cluster of its nearest non-noise object, ensuring that training examples are generated identically to those at inference time.

4.5 Distributed fit

Training examples are sharded by cluster, so a replica owns whole clusters and never needs a member it does not hold. Four replicas then fit the same network with `DistributedDataParallel` over gloo and gradients are averaged at every step.

4.6 Baselines

The *plain autoencoder* is DENCAD with $\hat{A} = I$: same architecture, hyperparameters, and score, fitted on single windows, which isolates the contribution of the graph. It is useful to determine if the neighbourhood graph structure is really providing value to the anomaly detection task. *K-Means* scores the distance to the nearest centroid, with k selected via validation. *DENCAST distance* uses density-based clustering but omits the autoencoder, scoring the distance to the closest clustered object. The *graph autoencoder* is DENCAD without the clustering: it applies the same network over the complete L-hop neighbourhood of the window, thereby matching the exact output that a full-graph convolution would produce at the centre.

Comparison We present threshold-free measures as our primary comparison, alongside window-level precision and recall, while reporting the number of covered anomalous episodes as a supplementary information.

4.7 Evaluation

All five anomaly detectors are thresholded by the same technique that uses the sigma rule. It is divied in three phases:

$$z(x) = \frac{s(x) - \mu}{\sigma}, \qquad \text{flag when } z(x) > k. \tag{7}$$

Table 2: Test average precision, best per row in bold. Five detectors, six processed datasets, one seed.

	DENCAD	graph AE	plain AE	DENCAST σ	K-Means
SWaT	0.8484	0.8499	**0.8545**	0.8197	0.8340
HAI	0.5734	0.5786	**0.5972**	0.5245	0.5532
ESA rare	0.2313	0.2307	**0.2424**	0.2280	0.2255
ESA rare, cont.	0.1708	**0.1907**	0.1671	0.1681	0.1623
ESA	**0.0533**	0.0384	0.0409	0.0390	0.0315
ESA, cont.	**0.0263**	0.0245	0.0253	0.0247	0.0246

Table 3: Test ROC-AUC, best per row in bold. Five detectors, six processed datasets, one seed.

	DENCAD	graph AE	plain AE	DENCAST σ	K-Means
SWaT	0.9239	0.9257	**0.9277**	0.9066	0.9094
HAI	0.8393	**0.8525**	0.8406	0.8027	0.8174
ESA rare	0.6515	0.6510	**0.6670**	0.6543	0.6449
ESA rare, cont.	0.6341	**0.6519**	0.6405	0.6270	0.6098
ESA	0.6109	0.6132	0.5932	**0.6290**	0.6066
ESA, cont.	0.5965	0.6041	**0.6063**	0.5964	0.5936

In phase one, the model is fitted on the training set, μ and σ are measured on its training scores, and k is chosen among ten candidates between 0.5 and 4 via validation F_1; for the two autoencoders, the epoch count is selected in the same phase using early stopping on validation average precision, with a patience of 20 and a maximum of 200 epochs. In phase two, the model is refitted on the combined training and validation windows for that epoch count (normal or contaminated depending on the specific experiment), and μ and σ are recomputed on this larger set. Finally, in phase three, the test set is used to compute the final measures for the comparison.

The hyperparameter grids vary significantly in size, requiring different search strategies. K-Means has a single hyperparameter, the number of centroids $k \in \{2, 4, 8, 16, 32, 64\}$, and all six values are evaluated; the plain autoencoder comprises sixteen combinations and is likewise tested exhaustively. The remaining grids (DENCAST distance, DENCAD, and Graph Autoencoder) are too large; thus, due to computational constraints, fifteen configurations were sampled at random from each. In every case, the final configuration is selected based on validation average precision.

The searched axes are $b \in \{16, 24\}$, permutations $\in \{10, 20\}$, band $\in \{20, 50\}$, $w/\varepsilon \in \{1, 2\}$, $\texttt{min_pts} \in \{10, 5\}$ for the graph; hidden $\in \{64, 128\}$, latent $\in \{16, 32\}$, layers $\in \{2, 3\}$ and learning rate $\in \{10^{-3}, 3 \cdot 10^{-3}\}$ for the autoencoder. Batch size 128, patience 20, a ceiling of 200 epochs, four replicas, Adam and one seed are fixed.

Metrics. Average precision and ROC-AUC are reported as the primary comparison because they are threshold-free. Precision, recall, and F_1 are evaluated at the *window* level, while the number of anomalous segments covered at least once is reported separately.

Implementation. PySpark 3.5.9 on a local four-worker session with the PyTorch gloo backend, hosted on a single machine with four cores and 16 GB of memory. The pipeline comprises 66 DVC stages.

5 Results

Overall, autoencoder-based detectors slightly outperform distance-based methods (K-Means and DENCAST distance) across nearly all datasets under both metrics, while little to no performance difference is observed between DENCAST distance and K-Means.

Among the autoencoders, no clear overall winner emerges, and incorporating the graph does not always pay for itself: the plain autoencoder, which operates on single observations without graph structure, achieves the highest score on three of the six datasets under either metric. DENCAD achieves the highest average precision on both ESA variants that consider rare events as legitimate—where all detectors yield scores between 0.025 and 0.053—and its margin is substantially wider than indicated by ROC-AUC.

6 Conclusion

For anomaly detection over multivariate sensor telemetry from industrial plants and spacecraft, reconstruction-based models prove to be slightly better than distance-based methods.

No clear performance difference was observed between K-Means and DENCAST distance. Although DENCAST theoretically yields superior clustering, the distance to an individual cluster object might be less informative than the distance to a centroid; however, these explanations remain speculative.

The construction of a neighbourhood graph does not consistently justify its computational overhead, and its overall utility appears to be highly dataset-dependent. In this study, DENCAD does not exhibit a performance advantage that offsets the cost of graph building. However, this finding specifically pertains to graphs constructed via locality-sensitive hashing (LSH) in feature space; evaluating DENCAD on inherently graph-structured datasets remains an important avenue for future work.

One limitation bounds the comparison: the hyperparameter grids are explored unequally. While the plain autoencoder and K-Means algorithms exhaustively enumerate all their combinations, DENCAD samples fifteen at random due to the severe combinatorial explosion of its hyperparameter space. Consequently, DENCAD's current figures represent a lower bound on its potential performance, leaving open the possibility that a fully searched grid could close or reverse the gap. Although conducting an exhaustive search is computationally expensive and highlights DENCAD's vulnerability to hyperparameter explosion, evaluating the complete grid remains a worthwhile endeavor for future work.

All five detectors compute an *unweighted* Euclidean distance within the same feature space—whether to a centroid, a core object, or between a window and its reconstruction—meaning that while the context and model architecture varied, the aggregation method remained constant. Consequently, every feature contributes equally to the distance calculation. Scaling each feature's error by its typical magnitude during training is identified as an important modification for future work, and all five detectors should be re-evaluated under this weighted formulation.

References

[1] Roberto Corizzo, Gianvito Pio, Michelangelo Ceci, and Donato Malerba. DENCAST: distributed density-based clustering for multi-target regression. *Journal of Big Data*, 6:43, 2019.

[2] Martin Ester et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *KDD*, pages 226–231, 1996.

[3] Moses S. Charikar. Similarity estimation techniques from rounding algorithms. In *STOC*, pages 380–388, 2002.

[4] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *SoCG*, pages 253–262, 2004.

[5] Deepak Ravichandran, Patrick Pantel, and Eduard Hovy. Randomized algorithms and NLP: using locality sensitive hash functions for high speed noun clustering. In *ACL*, pages 622–629, 2005.

[6] Kaize Ding et al. Deep anomaly detection on attributed networks. In *SDM*, pages 594–602, 2019.

[7] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.

[8] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *NeurIPS*, pages 1024–1034, 2017.

[9] Wei-Lin Chiang et al. Cluster-GCN: an efficient algorithm for training deep and large graph convolutional networks. In *KDD*, pages 257–266, 2019. Towards a rigorous evaluation of time-series anomaly detection. In *AAAI*, pages 7194–7201, 2022. Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress. *IEEE Transactions on Knowledge and Data Engineering*, 2023.

[10] Aditya P. Mathur and Nils Ole Tippenhauer. SWaT: a water treatment testbed for research and training on ICS security. In *CySWater*, pages 31–36, 2016.

[11] Hyeok-Ki Shin et al. HAI 1.0: HIL-based augmented ICS security dataset. In *USENIX CSET*, 2020.

[12] Krzysztof Kotowski et al. European Space Agency benchmark for anomaly detection in satellite telemetry. *arXiv preprint arXiv:2406.17826*, 2024. Accurate, large minibatch SGD: training ImageNet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.